

Taller Práctico: Automatización de Análisis Bioinformático con Bash

Ejercicio 1. Explorando directorios y archivos

Objetivo

Practicar navegación y búsqueda de archivos.

Instrucciones

Crear la siguiente estructura:

```
mkdir proyecto  
cd proyecto  
  
mkdir datos resultados scripts
```

Responder:

1. ¿Qué comando muestra el directorio actual?
2. ¿Qué comando lista todos los archivos incluyendo ocultos?
3. ¿Cómo encontrar todos los archivos FASTQ dentro del proyecto?

Desafío

Crear 5 archivos FASTQ vacíos:

```
touch muestra{1..5}.fastq
```

Ejercicio 2. Variables y automatización básica

Objetivo

Utilizar variables para evitar repetir comandos.

Instrucciones

Crear las variables:

```
PROYECTO=analisis_ecoli  
MUESTRA=SRR123456
```

Mostrar:

```
echo $PROYECTO  
echo $MUESTRA
```

Crear automáticamente:

```
mkdir $PROYECTO
```

Preguntas

¿Qué ventajas tiene utilizar variables en un pipeline?

Ejercicio 3. Procesamiento masivo mediante bucles

Objetivo

Automatizar tareas repetitivas.

Instrucciones

Crear archivos:

```
touch muestra{1..10}.fastq
```

Mostrar sus nombres mediante:

```
for archivo in *.fastq  
do  
    echo $archivo  
done
```

Desafío

Renombrar todos los archivos agregando "_raw":

```
muestral.fastq
```

↓

```
muestral_raw.fastq
```

Ejercicio 4. Construcción de scripts reutilizables

Objetivo

Transformar comandos en scripts.

Instrucciones

Crear:

```
nano contar_fastq.sh
```

Contenido:

```
#!/bin/bash

echo "Número de archivos FASTQ:"
ls *.fastq | wc -l
```

Otorgar permisos:

```
chmod +x contar_fastq.sh
```

Ejecutar:

```
./contar_fastq.sh
```

Preguntas

1. ¿Qué significa `#!/bin/bash`?
2. ¿Por qué es importante dar permisos de ejecución?

Ejercicio 5. Diseño de un proyecto bioinformático reproducible

Objetivo

Construir una estructura estándar para proyectos de secuenciación.

Crear la siguiente estructura

```
mkdir -p proyecto_variantes/{\
00_documentacion,\
01_datos_crudos,\
02_qc,\
03_trimmed,\
```

```
04_referencia,\n05_mapeo,\n06_variantes,\n07_anotacion,\n08_resultados,\n09_logs,\nscripts}
```

Visualizar:

```
tree proyecto_variantes
```

Crear archivo README_PROYECTO.md

```
proyecto_variantes/
```

```
00_documentacion/\n    Protocolos, manuales y metadatos.\n\n01_datos_crudos/\n    Archivos FASTQ originales descargados desde SRA o entregados por el\n    secuenciador.\n    Nunca modificar.\n\n02_qc/\n    Resultados de FastQC y MultiQC.\n\n03_trimmed/\n    Lecturas filtradas y recortadas tras Trimmomatic o fastp.\n\n04_referencia/\n    Genoma de referencia.\n    Índices de BWA/Bowtie2.\n\n05_mapeo/\n    Archivos SAM/BAM.\n    BAM ordenados e indexados.\n\n06_variantes/\n    Archivos VCF generados por FreeBayes, bcftools o GATK.\n\n07_anotacion/\n    Resultados de SnpEff, VEP u otras herramientas de anotación.\n\n08_resultados/\n    Tablas finales, figuras e informes.\n\n09_logs/\n    Registros de ejecución y errores.\n\nscripts/\n    Scripts Bash utilizados en el proyecto.
```

Preguntas

1. ¿Por qué los datos crudos nunca deben modificarse?
 2. ¿Por qué es útil separar resultados intermedios y finales?
-

Ejercicio 6. Automatización de FastQC

Objetivo

Procesar múltiples muestras simultáneamente.

Instrucciones

Crear:

```
nano ejecutar_fastqc.sh
```

Contenido:

```
#!/bin/bash

for archivo in *.fastq.gz
do
    fastqc $archivo -o ../02_qc/
done
```

Preguntas

¿Qué ocurriría si existieran 100 muestras?

Ejercicio 7. Automatización de mapeo

Objetivo

Automatizar alineamientos.

Instrucciones

Escribir un script que ejecute:

```
bwa mem referencia.fasta muestra.fastq.gz
```

para todas las muestras disponibles.

Guardar los resultados en:

05_mapeo/

Ejercicio 8. Automatización de llamado de variantes

Objetivo

Generar VCFs para múltiples muestras.

Instrucciones

Utilizar un bucle que:

1. Tome cada BAM.
2. Ejecute FreeBayes o bcftools.
3. Genere un VCF por muestra.

Resultado:

06_variantes/

Ejercicio 9. Pipeline completo

Objetivo

Integrar todos los pasos previos.

Crear un script maestro

pipeline.sh

que ejecute secuencialmente:

1. Control de calidad
2. Trimming
3. Mapeo
4. Ordenamiento BAM
5. Indexación
6. Llamado de variantes
7. Anotación

Registrar todo en:

09_logs/

mediante:

```
pipeline.sh > pipeline.log 2>&1
```

Ejercicio 10 (Integrador Final)

Objetivo

Diseñar un pipeline reproducible para el análisis de una cepa de E. coli.

El pipeline debe:

- Crear automáticamente toda la estructura del proyecto.
- Generar el README.
- Ejecutar FastQC.
- Ejecutar trimming.
- Mapear contra la referencia.
- Generar BAM ordenado.
- Generar BAM indexado.
- Llamar variantes.
- Anotar variantes.
- Guardar logs.
- Generar un reporte final.

Pregunta final

Si un investigador recibe únicamente:

```
README_PROYECTO.md  
scripts/
```

¿Debería ser capaz de reproducir completamente el análisis? Justifique.